1. Manual de Identificación

1. Manual de Identificación ____ *1*

1.1. Introducción ____ **2**

1.2. Modelado Físico ____ **3**

1.3. Experimentos ____ **4**

1.3.1. Ensayos Preliminares ____ 5

1.3.2. Ensayos Principales ____ 9

1.4. Análisis de los Datos ____ **12**

1.4.1. Excitación ____ 13

1.4.2. Filtrado ____ 15

1.5. Selección de la Estructura del Modelo ____ **17**

1.6. Estimación del Modelo ____ **20**

1.6.1. Estimación de Modelos Lineales ____ 20

1.6.2. Análisis Espectral a partir de la Función de Covarianza ____ 24

1.6.3. Análisis Espectral a partir de la Transformada de Fourier Discreta ____ 26

1.6.4. Modelos Paramétricos ____ 28

1.6.5. Reducción del Modelo ____ 30

1.7. Validación ____ **32**

1.7.1. Criterio de Validación del Modelo ____ 33

1.7.2. Ubicación de Polos y Ceros ____ 35

1.7.3. Diagramas de Bode ____ 37

1.7.4. Análisis de Residuos ____ 38

1.7.5. Simulación y Validación Cruzada ____ 41

1.8. Bibliografía ____ **43**

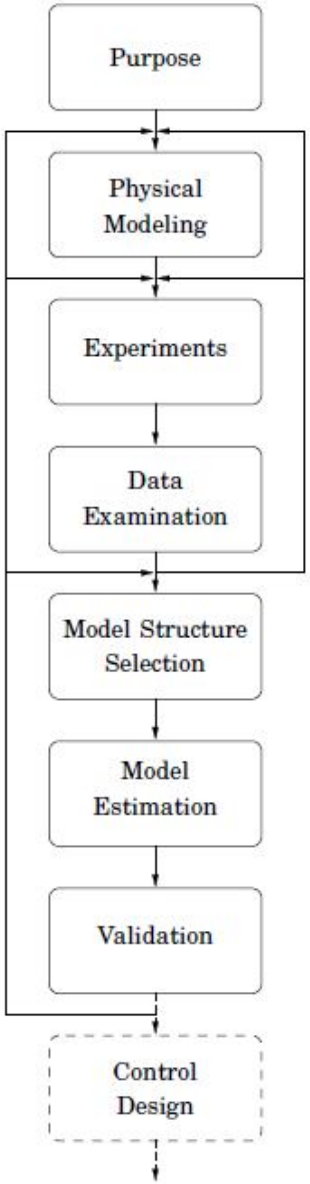
1.1. Introducción

Definir el *propósito* de la Identificación

- Control
- Simulación
- Predicción
- Detección de fallas

El propósito del modelo influye en la elección del método de identificación y de los experimentos a realizar.

Si el modelo será utilizado para diseñar un controlador, es importante que tenga buena precisión en los alrededores de la *frecuencia de cruce*.



1.2. Modelado Físico

En muchos casos se puede obtener un modelo a partir de las Leyes Físicas.

Pero, en general queda en función de **parámetros** desconocidos.

Si algunos **parámetros se conocen y otros no**, conviene **no estimar** los parámetros conocidos (no siempre es posible).

Generalmente, la **estructura del modelo** se obtiene a partir de las Leyes Físicas.

- Orden del sistema
- Presencia de integradores
- Frecuencia de resonancia

Si no se conoce nada acerca del proceso, se utiliza la **identificación de caja negra**

Si se conoce algo, se utiliza la **identificación de caja gris**

1.3. Experimentos

Se deben realizar en dos etapas:

Etapa 1:

- Ensayos preliminares. Impulso, escalón
- Para obtener un conocimiento primario
- Visualizar ganancia estática, principales constantes de tiempo, presencia de retardos o integradores
- Verificar si es dependiente del tiempo y si existen perturbaciones
- Todo esto contribuye al diseño de los **ensayos principales**

Etapa 2:

- Ensayos principales
- Nos entregarán los datos para utilizar con las rutinas de identificación

1.3.1. Ensayos Preliminares

- **Invariancia en el tiempo**
  - Muchos métodos de identificación asumen que el proceso es invariante en el tiempo
  - Hacer ensayos a diferentes tiempos
  - A veces un proceso parece variar en el tiempo
    - Se puede deber a dinámicas muy lentas en donde se ve el sistema en distintos puntos en el tiempo
    - Si se quieren identificar dinámicas rápidas
      - Probar con ensayos más largos en el tiempo o
      - eliminar las tendencias o valores estacionarios (*trends*).
  - Si efectivamente es variante en el tiempo
      - utilizar **rutinas recursivas**

- **Linealidad**
  - Muchos de los algoritmos de identificación obtienen modelos lineales
  - Se debe analizar la linealidad
    - Ensayar el proceso para diferentes amplitudes de la entrada
    - Para una onda cuadrada a la entrada, respuesta debería ser simétrica
    - Obtener el espectro de coherencia
  - La mayoría de los sistemas son no-lineales
  - Es posible obtener un modelo lineal con variaciones pequeñas de entrada
  - En este caso, cuidado con el tamaño del ruido o las perturbaciones

- Hay no linealidades que se eliminan sumando alguna señal. Por ejemplo:
    - Mover un motor en una única dirección.
    - Agregar una vibración de relativamente alta frecuencia para eliminar el rozamiento.
    - Cambiar el estado estacionario
- Por el contrario, para eliminar la fricción, conviene tener entradas de mayor amplitud

- **Análisis de Transitorios**
  - La respuesta al escalón o impulso dan información de
    - Ganancia estática
    - Principales constantes de tiempo
    - Retardos (no siempre)
    - Comportamientos de no mínima fase
  - Puede ser un escalón en la entrada o en una perturbación
- **Respuesta en Frecuencia**
  - Nos informa acerca de:
    - Ganancia estática
    - Polos y ceros
    - Retardos

1.3.2. Ensayos Principales

- De aquí van a salir los datos para utilizar en las rutinas de identificación
- La identificación entrega un buen resultado en aquellas frecuencias en donde la entrada tiene mayor energía. (la entrada tiene buena excitación a esas frecuencias)
- Es conveniente definir a qué frecuencias se desea tener menor error
- Una secuencia binaria pseudo aleatoria (PBRS) es en general una buena elección. Se define con:
    - Período de muestreo (T)
    - Cantidad de muestras para el corrimiento de la secuencia (M)
    - Cantidad de datos recogidos (N)
    - Amplitud de la señal (A)

- Recetas
  - 1/(TxM) = ancho de banda del sistema
  - Para evitar aliasing: M= 2 – 10 (se introduce un nuevo período que es TM que puede resultar muy grande)
  - Tener entre 5 y 10 muestras durante el período de crecimiento
  - La duración del experimento debería ser entre 5-10 veces la mayor constante de tiempo a estimar.

- Dividir los datos en tres:
  - Datos con transitorios
  - Datos para identificación y
  - Datos para validación
- Amplitud de la entrada
  - La mayor posible para minimizar el efecto de fricciones y ruido
  - Evitar alejarse demasiado del punto sobre el que se linealiza
  - En general, el punto de equilibrio no coincide con u=0

ATENCIÓN A ESTO
SI HACEMOS EXPERIMENTOS

1.4. Análisis de los Datos

Tendremos columnas de u e y

Conviene dividirlo en dos conjuntos

- Identificación y
- Validación

```
zi=iddata(y(1:N1),u(1:N1),h);
zv=iddata(y(N1+1:N),u(N1+1:N),h);
```

Hacer un gráfico y verificar (en ambos conjuntos):

- Muestras *falsas.* Sacarlas directamente
- Aliasing: hay que modificar el experimento
- Tendencias: eliminarlos
  - Constante: `zi=detrend(zi,'constant')`
  - Lineal: `zi=detrend(zi);`

1.4.1. Excitación

Debe proveer suficiente energía en las frecuencias de interés

Estudiar el espectro tanto de la entrada como de la salida

Deben estar presentes las frecuencias de interés

Estudiar las condiciones de excitabilidad de la entrada

Se puede filtrar u e y para concentrarse en el rango útil de frecuencias

Si la señal de entrada no tiene las componentes adecuadas, hay que repetir el experimento con otra entrada.

```
yi=zi.OutputData; ui=zi.InputData;
spectrum(ui,yi)
S=spectrum(ui,yi);
```

```
S=[Suu Syy Suy Tyu Gamuy Suuc Syc Suyc]
  Suu=u-vector power spectral density
  Syy=y-vector power spectral density
  Suy=Cross spectral density
  Tuy=Complex transfer function Suy./Suu
  Gamuy=Coherence function
     (abs(Suy).^2)./(Suu.*Syy)
  Suuc,Syyc,Suyc=Confidence ranges
```

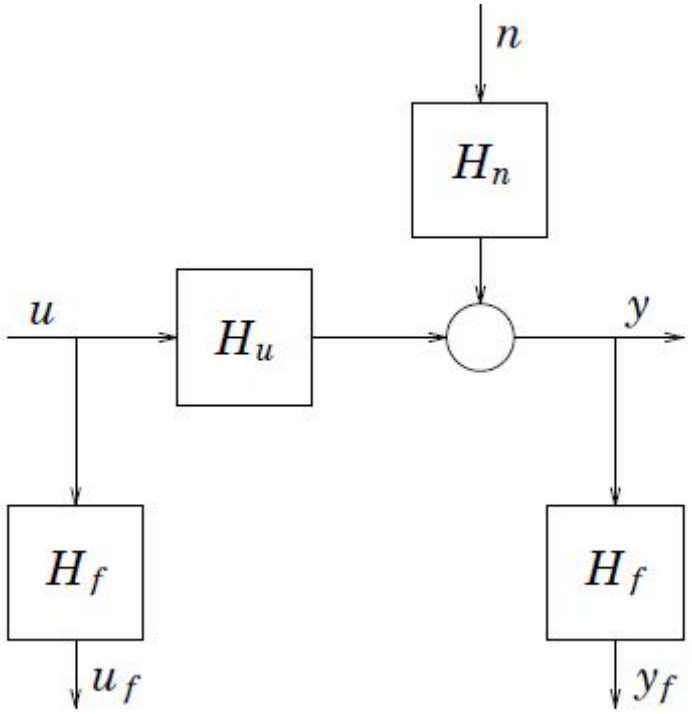
`psd`	Power spectral density
`csd`	Cross spectral density
`cohere`	Coherence function estimate
`tfe`	Transfer function estimate
`spectrum`	`psd`, `csd`, `cohere`, `tfe` combined

1.4.2. Filtrado

En general, es bueno filtrar con un pasa bajos o un pasa banda, los valores de u e y

Se concentra el análisis en un rango de frecuencias

El ruido tiene presencia en altas frecuencias



Filtro Butterworth:

```
zif=idfilt(zi,N,Wn);
```

Con

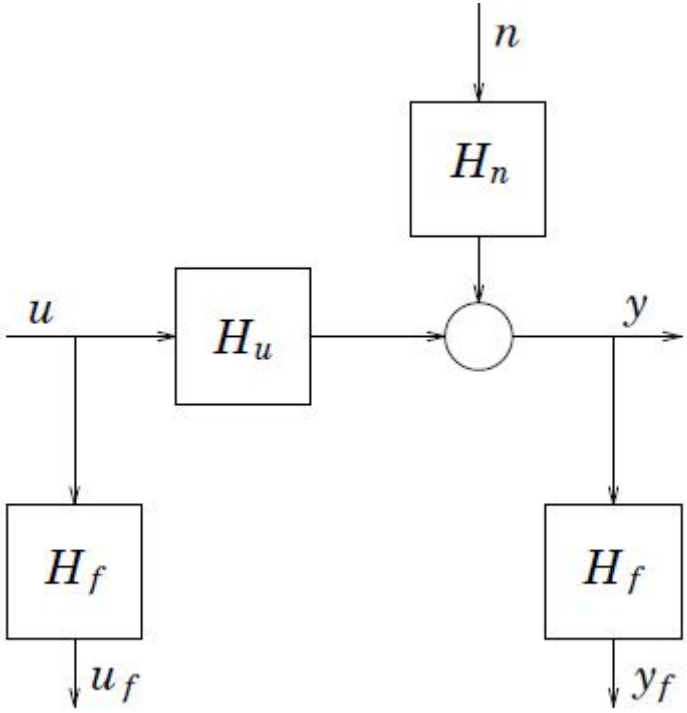
```
N=Order of Butterworth filter
Wn=Cut-off frequencies in fractions
   of the Nyquist frequency.
     Wn scalar gives lowpass filter
     Wn=[Wl Wh] gives bandpass filter
```

Este asegura que no hay distorsión de fase

Cuidado que el filtro afecta el modelo del ruido

En el gráfico, si queremos H_n en base a y_f

Obtendremos $H_n H_f$



1.5. Selección de la Estructura del Modelo

La estructura del modelo indica la cantidad y la función de los parámetros a identificar

Modelo simple

$$y_k = Ku_k$$

Complejidad del modelo vs precisión

Modelo más general

$$A(q)y_k = \frac{B(q)}{F(q)}u_{k-d} + \frac{C(q)}{D(q)}e_k$$

$$A(q) = 1 + a_1 q^{-1} + \cdots + a_{n_a} q^{-n_a}$$

$$B(q) = b_1 + b_2 q^{-1} + \cdots + b_{n_b} q^{-n_b+1}$$

$$C(q) = 1 + c_1 q^{-1} + \cdots + c_{n_c} q^{-n_c}$$

$$D(q) = 1 + d_1 q^{-1} + \cdots + d_{n_d} q^{-n_d}$$

$$F(q) = 1 + f_1 q^{-1} + \cdots + f_{n_f} q^{-n_f}$$

Raramente se usa la estructura general

Casos Particulares

Modelo AR

$$A(q)\,y_k = e_k$$

Solo tiene entrada de ruido

Modelo ARX

$$A(q)\,y_k = B(q)u_{k-d} + e_k$$

Modelo ARMAX

$$A(q)\,y_k = B(q)u_{k-d} + C(q)e_k$$

Modelo de Error de Entrada Salida (OE)

$$y_k = \frac{B(q)}{F(q)}u_{k-d} + e_k$$

Modelo Box- Jenkins

$$y_k = \frac{B(q)}{F(q)} u_{k-d} + \frac{C(q)}{D(q)} e_k$$

La elección depende del ruido

No necesariamente un modelo con más parámetros o más grados de libertad es un mejor modelo

1.6. Estimación del Modelo

Identificación del Sistema: procedimiento para encontrar un modelo a partir de los datos.

Estimación del Modelo: procedimiento para ajustar un modelo de una determinada estructura.

1.6.1. Estimación de Modelos Lineales

En esta clase de modelos comprendemos al análisis de

- Covarianza y
- Espectro

No se requiere fijar una estructura del modelo (orden de los polinomios, retardo, etc)

- **Análisis de Correlación**

  Dos comandos básicos: `cra` y `covf`

`cra`	Impulse response from correlation analysis
`covf`	Covariance function estimate

La función de covarianza se estima,

$$\hat{C}_{yu}(\tau) = \frac{1}{N}\sum_{k=1}^{N} y(k+\tau)u(k)$$

$$\hat{C}_{uu}(\tau) = \frac{1}{N}\sum_{k=1}^{N} u(k+\tau)u(k)$$

$$\hat{C}_{yy}(\tau) = \frac{1}{N}\sum_{k=1}^{N} y(k+\tau)y(k)$$

La estimación de la respuesta impulsional se extrae de la relación

$$\hat{C}_{yu}(k) = \sum_{l=0}^{\infty} h(l)\hat{C}_{uu}(k-l)$$

Suponiendo que la entrada es ruido blanco,

$$\hat{h}(k) = \frac{1}{\sigma_u^2}\hat{C}_{yu}(k)$$

Si la entrada no es ruido blanco, se puede utilizar un filtro blanqueador H_w de modo que $u_f = H_w u$ sea lo suficientemente blanco.

El comando `cra` utiliza estos filtros para calcular la respuesta impulsional:

```
ir=cra(zi)
```

La respuesta al escalón se puede obtener con:

```
sr=cumsum(ir)
plot(sr)
```

1.6.2. Análisis Espectral a partir de la Función de Covarianza

La Transformada Discreta de Fourier Filtrada de las covarianzas da una estimación de los espectros.

Por ejemplo

$$\hat{S}_{yu}(j\omega) = \sum_{\tau=-M}^{M} \hat{C}_{yu}(\tau) W_M e^{-j\omega\tau}$$

Donde W_M es una función ventana que corrige los sesgos de la estimación.

La estimación depende mucho de esta ventana.

La función spa estima la función de transferencia (en forma de diagrama de Bode) y el espectro del ruido:

$$\hat{H}\left(e^{j\omega}\right) = \frac{\hat{S}_{yu}(j\omega)}{\hat{S}_{uu}(j\omega)}$$

$$\hat{S}_{nn}(j\omega) = \hat{S}_{yy}(j\omega) - \frac{\left|\hat{S}_{yu}(j\omega)\right|^2}{\hat{S}_{uu}(j\omega)}$$

Una secuencia típica de comandos sería

```
[H,Snn]=spa(zi,M,w);
bode(H)
bode(Snn)
```

spa	Spectral analysis

1.6.3. Análisis Espectral a partir de la Transformada de Fourier Discreta

El comando `etfe` calcula la función de transferencia como cociente de las DTFs de la salida y entrada.

$$\hat{H}\left(e^{j\omega_k}\right)=\frac{\hat{Y}\left(e^{j\omega_k}\right)}{\hat{U}\left(e^{j\omega_k}\right)}$$

En Matlab

```
H=etfe(zi);
bode(H)
```

Igual que `spa`, utiliza una ventana.

Varía mucho con la elección de la ventana

No es fácil encontrar la mejor ventana.

Otra forma de calcularlo es

```
H=tfe(ui,yi);
```

`etfe`	DFT transfer function estimate
`tfe`	Welch's transfer function estimate
`spectrum`	Various power spectra using Welch's method
`fft`	Discrete Fourier transform

1.6.4. Modelos Paramétricos

Se supone que se seleccionó una estructura de modelo

Falta definir el orden y estimar los parámetros de los polinomios

Todas las funciones de Matlab tienen la forma,

```
model=functionname(zi,orders)
```

Por ejemplo

```
na=1; nb=1; nc=1; nd=1; nf=1; nk=1;
orders=[na nb nc nd nf nk];
pem1=pem(zi,orders)
```

La información se presenta,

```
present(pem1);
```

Para Modelos ARX hay dos métodos:

- Mínimos cuadrados y
- Variables instrumentales

Las otras estructuras de modelos usan métodos de predicción del error

`pem`	Estimate general model
`ar`	Estimate AR model
`ivar`	IV estimate of AR model
`iv4`	IV estimate of AR model
`arx`	LS estimate of ARX model
`ivx`	IV estimate of ARX model
`armax`	Estimate ARMAX model
`oe`	Estimate output-error model
`bj`	Estimate Box-Jenkins model
`present`	Presentation of estimated model

1.6.5. Reducción del Modelo

Esto es una alternativa a la estimación convencional.

Primero se estima un modelo ARX de orden elevado y luego se reduce el modelo

Un modelo de mayor orden extrae más información de los datos

En la reducción, nos quedamos con la parte más importante

Una vía es hacer

- una *realización en espacios de estado balanceada.*
- El gramiano de esta realización marca los estados de importancia
- Luego, se toman solo los estados significativos

Secuencia típica de comandos:

```
arx1=arx(zi,[10 9 1]);
[A,B,C,D]=ssdata(arx1);
[Ab,Bb,Cb,M,T]=dbalreal(A,B,C);
```

resultando

```
M=[21 12 12 0.03 0.03 0.02 0.02 0 0 0]
```

Los últimos 7 estados se pueden eliminar con los sgtes comandos:

```
[Ab,Bb,Cb,Db]=dmodred(Ab,Bb,Cb,D,4:10);
[b,a]=ss2tf(Ab,Bb,Cb,Db);
arx1red=idpoly(a,b);
```

`ssdata`	Conversion to state-space
`tfdata`	Conversion to transfer function
`dbalreal`	Discrete balanced realization
`balreal`	Continuous balanced realization
`dmodred`	Discrete model order reduction
`modred`	Continuous model order reduction
`ss2tf`	State-space to transfer function conversion
`idpoly`	Polynomial to transfer function conversion

1.7. Validación

Hay diferentes caminos

- Polos-ceros
- Diagramas de Bode
- Análisis de residuos
- Simulación
- Validación cruzada

Es conveniente utilizarlos a todos

1.7.1. Criterio de Validación del Modelo

Se puede analizar algún índice con diferentes modelos

El comando present da dos índices:

- Función de costo y
- FPE: Akaike's Final Prediction Error

Otra forma más sofisticada es utilizar los comandos arxstruc e ivstruc junto a selstruc

Ejemplo

```
NN=[2 1 1;3 1 1;3 2 1;4 3 1;5 4 1];
V=arxstruc(zi,zv,NN);
selstruc(V);
```

Cada columna de NN contiene el orden de un modelo ARX

El comando `arxstruc` estima los parámetros y el costo de cada modelo definido en NN basado en los datos zi y validado con los datos zv

`selstruc` grafica el costo en función de la cantidad de parámetros

El comando `ivstruc` se utiliza del mismo modo

`arxstruc`	Loss functions for families of ARX models
`ivstruc`	Output-error fit for families of ARX models
`selstruc`	Select model structures according to various criteria
`struc`	Generate typical structure matrices for `arxstruc` and `ivstruc`

1.7.2. Ubicación de Polos y Ceros

Una idea básica de la ubicación de los polos y ceros se obtiene de los experimentos preliminares y de los modelos no paramétricos

Es bueno verificar que los polos y ceros del modelo paramétrico es coherente con lo previo.

Es bueno detenerse en el análisis del modelo del ruido.

Si este tiene frecuencias predominantes, deberían aparecer polos resonantes a esas frecuencias

Una visualización de los polos y cero, puede mostrar que el orden del modelo es muy grande

O que hay polos muy cercanos a ceros, con lo que se puede pensar en una reducción del modelo

```
pzmap(pem1);
```

`zpkdata`	Zeros, poles, static gains and their standard deviations
`pzmap`	Plot of zeros and poles

1.7.3. Diagramas de Bode

Algo similar se puede verificar a partir de los diagramas de Bode

- Polos y ceros dominantes
- ganancia estática

Igualmente con el diagrama de Nyquist

El espectro del ruido se puede obtener agregando un segundo argumento de salida en `th2ff`

```
H=idfrd(pem1('m'));
bode(H);
Snn=idfrd(pem1('n'));
bode(Snn);
```

idfrd	Frequency function for the model

1.7.4. Análisis de Residuos

Los modelos paramétricos analizados tienen la forma

$$y_k = H_u(q)u_k + H_e(q)e_k$$

Donde las funciones de transferencias son racionales

Los residuos se computan como

$$\varepsilon_k = H_e^{-1}(q)\left[y_k - H_u(q)u_k\right]$$

Si los residuos se computan con los datos utilizados para la identificación, deberían ser, en teoría, ruido blanco e independiente de la entrada.

Si no, se puede deber a un inadecuado

- orden de los polinomios
- estructura
- cantidad de datos

Los residuos se pueden analizar de distintas maneras:

- autocorrelación de residuos
- correlación cruzada entre residuos y entrada y

Se pueden visualizar con

```
resid(pem1,zi);
```

Los residuos se deben analizar con los datos para identificación y con los datos para validación.

Los residuos con datos para validación, en teoría, deberían ser también ruido blanco.

Manejarlos con cuidado

Hay métodos que hacen foco en la relación entrada salida (por ejemplo VI), es decir en $H_u(q)$.

En estos casos es probable que los residuos no sean ruido blanco

No es necesario utilizar los modelos reducidos para este análisis

`resid`	Computes and tests the residuals associated with a model

1.7.5. Simulación y Validación Cruzada

Es útil para ver cómo el modelo reproduce la salida frente a una entrada real.

Ejemplo

```
ui=zi.InputData;
yi=zi.OutputData;
ys=idsim(ui,pem1);
plot([yi ys]);
```

Más interesante es utilizar *zv* para ver si efectivamente el modelo reproduce las dinámicas principales

```
uv=zv.InputData;
yv=zv.OutputData;
ys=idsim(uv,pem1);
plot([yv ys]);
```

Se puede verificar el modelo completo utilizando los residuos como ruido

```
zv=iddata(yv,uv,h);
ev=resid(pem1,zv);
yc=idsim([uv ev.OutputData],pem1);
plot([yv yc]);
```

Estos análisis son, probablemente los más importante, porque verifican el objetivo para el que fue calculado el modelo

`compare`	Compare simulated predicted or output with the measured output
`idsim`	Simulate a given system
`pe`	Prediction errors
`predict`	M-step ahead prediction
`resid`	Compute and test the residuals associated with a model

1.8. Bibliografía

Johansson, R. (1993): *System Modeling and Identification*. Prentice-Hall, Englewood Cliffs, New Jersey.